SISTEMAS MULTIDIMENSIONALES

Práctica 1: Diseño de Sistemas Multidimensionales

Francisco Javier Santos Rivas

Índice

1. Análisis del sistema real **3**
2. Diseño previo **4**
2.1 Definición de la empresa ficticia 4
2.2 Fuentes de datos OLTP a integrar 5
2.3 Preguntas de negocio 6
2.4 Decisiones estratégicas 6
2.5 Modelado OLAP conceptual 7
2.6 Bus Matrix conceptual 8
2.7 Diagramas OLAP iniciales 9
3. Diseño de fuentes de datos transaccionales y población inicial **10**
3.1 Introducción 10
3.2 Arquitectura de esquemas OLTP 10
3.3 Diseño por departamento 11
3.3.1 Departamento Producción Agrícola (oltp_produccion) 11
3.3.2 Departamento Calidad y Trazabilidad (oltp_calidad) 12
3.3.3 Departamento Ventas Comerciales (oltp_ventas) 13
3.3.4 Departamento Logística y Exportación (oltp_logistica) 14
3.3.5 Departamento Finanzas y Costes (oltp_finanzas) 16
3.4 Relaciones cruzadas entre departamentos 18
3.5 Población de datos sintéticos (DML) 18
4. Modelo dimensional y ETL básico **19**
4.1 Metodología de transformación OLTP 20
4.2 Mapeo de fuentes OLTP a dimensiones y hecho 21
4.3 Bus Matrix final 22
4.4 Modelo estrella propuesto 22
4.5 Scripts de creación del esquema estrella 23
4.6 Carga de dimensiones 26
4.7 ETL básico de la tabla de hechos 27
4.8 Propuesta de modelo snowflake 29
5. Benchmarking y evaluación **30**
5.1 Objetivo del benchmarking 30
5.2 Consultas que resuelven las preguntas de negocio 30
5.3 Ejemplo de roll-up y drill-down 35
5.4 Script Python de benchmarking 35
5.5 Resultados del benchmarking 36
5.6 Interpretación de resultados 36
6. Conclusiones **37**
Anexo A. Archivos adjuntos entregados **38**

1. Análisis del sistema real

El sistema real seleccionado es INCLASNS (Indicadores Clave del Sistema Nacional de Salud), una aplicación interactiva del Ministerio de Sanidad que publica indicadores sanitarios y de contexto para apoyar el análisis y la toma de decisiones dentro del Sistema Nacional de Salud.

Objetivo del negocio: Entender la salud de los ciudadanos, el funcionamiento del sistema sanitario público, los factores que influyen en la salud y observar cambios y tendencias (desde 1990).

Fuentes de datos: INCLASNS integra información procedente de múltiples registros y operaciones estadísticas oficiales, entre ellas Defunciones según causa de muerte, Cifras de población, Estadística de Gasto Sanitario Público, Sistema de Cuentas de Salud, Sistema de Información de Atención Primaria, Sistema de Información de Atención Especializada, Sistema de Información de Listas de Espera, Sistema de Información de Consumo Farmacéutico, SINAC, Organización Nacional de Trasplantes y otras fuentes del Ministerio, el INE y organismos públicos relacionados.

Dimensiones principales (con jerarquías):

- Tiempo: año.
- Territorio: país > comunidad autónoma.
- Población: sexo y, en algunos indicadores, nivel socioeconómico.
- Indicador: dimensión temática > dominio > indicador.
- Fuente: organismo o sistema de información de origen.

Hechos y métricas clave: el hecho principal es la observación de un indicador sanitario en un contexto temporal, territorial y poblacional determinado. La métrica principal es el valor del indicador, que según el caso puede expresarse como porcentaje, tasa, número absoluto, gasto, tiempo de espera, prevalencia, incidencia o esperanza de vida.

Tecnología utilizada: la parte visible del sistema es una aplicación web interactiva con fichas, informes y documentación metodológica accesible desde el portal del Ministerio de Sanidad. No se identifica públicamente el motor OLAP o la base de datos subyacente, por lo que sólo puede afirmarse con seguridad la existencia de una capa de consulta y visualización web.

Preguntas de negocio que el sistema puede resolver:

- ¿Cómo evoluciona la esperanza de vida por comunidad autónoma y año?
- ¿Qué diferencias existen entre territorios en accesibilidad, uso de recursos y listas de espera?
- ¿Qué desigualdades por sexo o nivel socioeconómico aparecen en determinados indicadores?
- ¿Qué tendencias históricas muestran los indicadores esenciales del SNS?
- ¿Qué relación puede observarse entre recursos, gasto y resultados en salud en distintas regiones?

2. Diseño previo

2.1 Definición de la empresa ficticia

La empresa se denomina *AgroCeuta*. Se dedica a la producción, clasificación, envasado y exportación de productos agrícolas hacia mercados nacionales e internacionales, especialmente del área mediterránea y de la Unión Europea. Su actividad principal gira en torno a cultivos de alto valor comercial, como tomate, pimiento, aguacate, cítricos y frutos rojos, entre otros, operando tanto con fincas propias como con productores asociados.

AgroCeuta gestiona una operativa compleja porque combina procesos agrícolas, control de calidad por lotes, trazabilidad alimentaria, certificaciones fitosanitarias, logística de exportación, costes de manipulación, incidencias en transporte y análisis financiero por campaña.

La empresa trabaja por campañas agrícolas, ya que la disponibilidad, calidad, rendimiento y precio de los productos dependen de la época del año, del clima, de la finca de origen y del mercado de destino. Una parte importante del negocio consiste en decidir qué productos cultivar, cómo clasificarlos, qué clientes priorizar y qué rutas logísticas utilizar para maximizar el margen y reducir mermas.

Actualmente, cada área funcional utiliza su propio sistema transaccional y la información está fragmentada: producción registra cosechas y parcelas; calidad controla lotes, calibres e incidencias; ventas gestiona pedidos y clientes; logística organiza envíos y exportaciones; y finanzas registra

costes, facturas y cobros. Esta separación dificulta el análisis conjunto y la toma de decisiones estratégicas.

2.2 Fuentes de datos OLTP a integrar

Para construir el futuro Data Warehouse, AgroCeuta deberá integrar la información procedente de cinco sistemas distintos. Cada uno representa un área del negocio y aporta una parte necesaria del análisis multidimensional.

Departamento	**Sistema OLTP**	**Entidades principales**	**Volumen aprox.**
Producción agrícola	oltp_produccion	fincas, parcelas, campañas, cultivos	350 cosechas/año
Calidad	oltp_calidad	lotes, controles_calidad, calibres, certificados, incidencias_calidad	1500 controles/año
Ventas	oltp_ventas	clientes, contratos, pedidos, detalle_pedido, tarifas	950 pedidos/año
Logística	oltp_logistica	envios, transitarios, puertos, aduanas, incidencias_envio	900 envíos/año
Finanzas	oltp_finanzas	facturas, cobros, costes_lote, costes_logisticos, tipos_cambio	950 facturas/año

Descripción breve de cada sistema:

- oltp_produccion: registra la actividad agrícola en campo: qué se cultiva, en qué finca y parcela, en qué campaña y con qué rendimiento.
- oltp_calidad: almacena los controles realizados sobre cada lote, incluyendo calibre, categoría comercial, incidencias, rechazos y certificaciones.

- oltp_ventas: contiene la relación con clientes, pedidos, líneas de pedido, precios pactados y condiciones comerciales.
- oltp_logistica: gestiona la expedición de mercancías, los transitarios, los puertos de salida, los tiempos de tránsito y las incidencias durante el transporte.
- oltp_finanzas: registra el impacto económico del negocio - costes de producción, costes logísticos, facturación, cobros y márgenes.

2.3 Preguntas de negocio

El Data Warehouse de AgroCeuta deberá responder a las siguientes preguntas analíticas:

1. ¿Cuál es la evolución de ingresos, costes y margen por producto, variedad, país de destino y trimestre a lo largo de las distintas campañas?
2. ¿Qué fincas de origen generan mayor volumen de ventas y mejor margen neto según producto y campaña?
3. ¿Qué clientes y mercados internacionales aportan mayor volumen de negocio, mejor precio medio por kilogramo y mayor margen?
4. ¿Qué transitarios, rutas logísticas y puertos de salida ofrecen menores tiempos de entrega y menor coste logístico por kilogramo exportado?
5. ¿Cómo influye la calidad del lote de origen (calibre, categoría comercial, certificados) en el precio final de venta y en el margen obtenido?

2.4 Decisiones estratégicas

A partir de las preguntas anteriores, AgroCeuta podrá tomar decisiones como las siguientes:

- Decisión 1 - Optimización del mix de producción y comercialización: decidir qué cultivos y variedades ampliar, mantener o reducir en función de su rendimiento en finca, calidad obtenida y margen comercial por mercado de destino.
- Decisión 2 - Reorganización comercial y logística: decidir qué clientes priorizar, qué transitarios contratar y qué rutas utilizar según costes, tiempos de entrega, incidencias y rentabilidad final.

También podrían derivarse otras decisiones complementarias:

- Reasignar fincas y parcelas a cultivos más rentables.
- Ajustar precios o condiciones comerciales por cliente o país.
- Introducir mejoras de calidad en productos con alta merma.
- Rediseñar el flujo logístico en campañas con saturación o retrasos.

2.5 Modelado OLAP conceptual

El modelado OLAP conceptual de AgroCeuta parte de las cinco preguntas de negocio definidas previamente y sigue el enfoque de Kimball: primero se identifica el proceso de negocio, después la granularidad, a continuación las dimensiones y, por último, las métricas. En esta fase el objetivo no es todavía implementar el Data Warehouse definitivo, sino identificar las perspectivas de análisis y los posibles hechos candidatos que darán lugar al modelo dimensional final.

A partir de las preguntas de negocio, se identifican dos procesos de negocio candidatos:

- Exportaciones: proceso principal orientado al análisis de ventas internacionales, costes, márgenes, logística y comportamiento de clientes.
- Controles de calidad: proceso complementario orientado al análisis de lotes, calibres, categorías comerciales, incidencias y certificados.

Las dimensiones candidatas del sistema son las siguientes:

- Dim_Tiempo: fecha, día, mes, trimestre, año, día_semana.
- Dim_Producto: nombre_producto, variedad, familia_producto.
- Dim_Geografia: ciudad, provincia, región, país, continente.
- Dim_Cliente: nombre_cliente, tipo_cliente, canal, segmento.
- Dim_Finca: nombre_finca, parcela, municipio, provincia, zona_agricola.
- Dim_Calidad: categoria_comercial, calibre, tipo_certificado.
- Dim_Logistica: transitario, puerto_salida, puerto_llegada, tipo_transporte, ruta_logistica.
- Dim_Campania: nombre_campania, fecha_inicio, fecha_fin, anio_agricola.

Los hechos candidatos identificados son:

- Fact_Exportaciones: una fila por línea de pedido exportada.
- Fact_Controles_Calidad: una fila por control de calidad realizado sobre un lote.

La granularidad conceptual del hecho principal Fact_Exportaciones es: una fila por línea de pedido exportada, correspondiente a un lote concreto, vendido a un cliente en una fecha determinada, procedente de una parcela/finca específica, asociado a una campaña agrícola y expedido mediante una ruta logística concreta.

Las métricas candidatas de Fact_Exportaciones son:

- kilos_vendidos
- cajas_vendidas
- importe_bruto
- descuento
- importe_neto
- coste_producto
- coste_logistico
- coste_total
- margen
- precio_medio_kg
- tiempo_entrega_dias

La granularidad conceptual del hecho secundario Fact_Controles_Calidad es: una fila por control de calidad aplicado a un lote de origen en una fecha determinada.

Las métricas candidatas de Fact_Controles_Calidad son:

- kilos_inspeccionados
- kilos_rechazados
- porcentaje_rechazo
- puntuacion_calidad

2.6 Bus Matrix conceptual

La siguiente Bus Matrix resume la relación entre los procesos de negocio candidatos y las dimensiones identificadas:

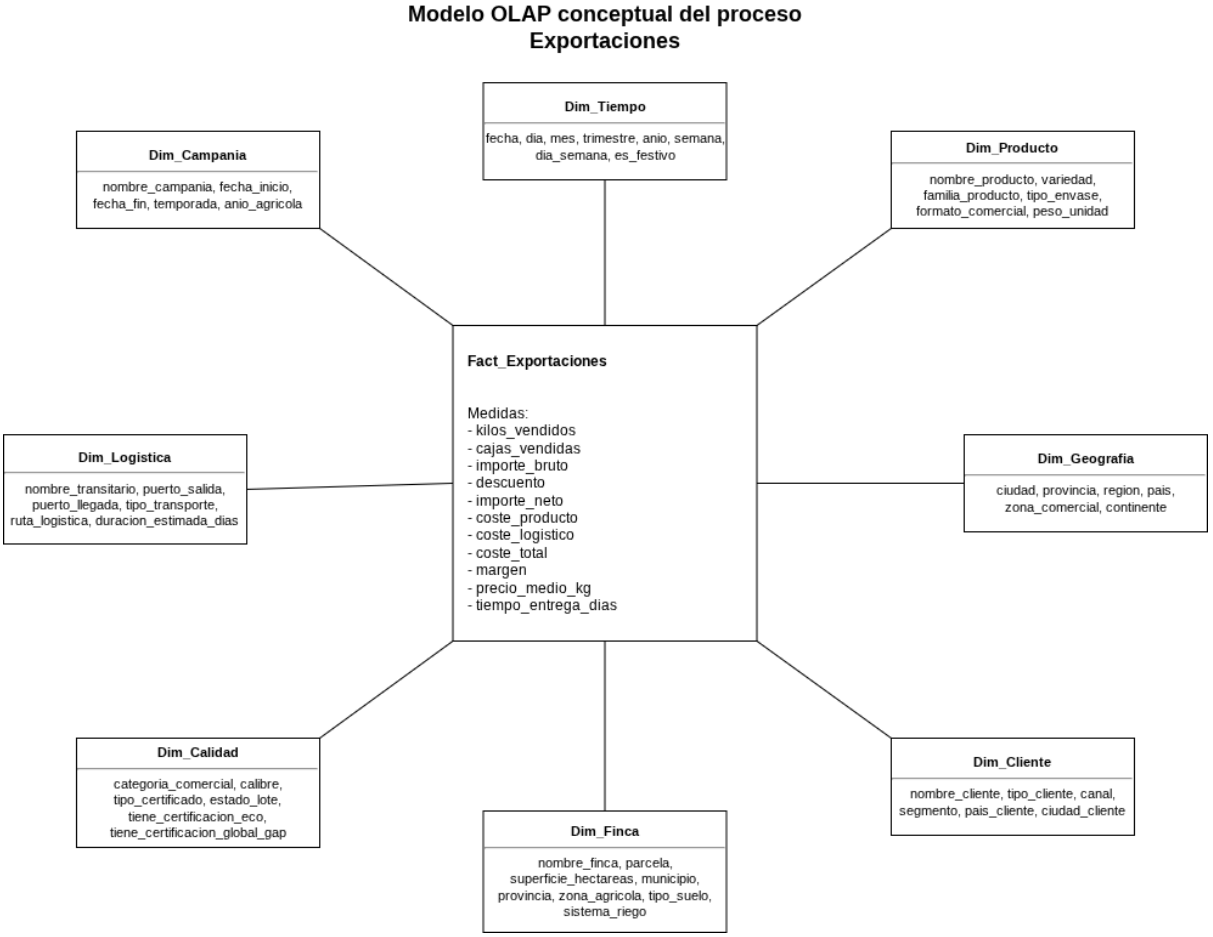
Proceso	Tiempo	Producto	Geografía	Cliente	Finca	Calidad	Logística	Campaña
Exportaciones	X	X	X	X	X	X	X	X
Controles de calidad	X	X			X	X		X

Esta matriz muestra que las dimensiones Tiempo, Producto, Finca, Calidad y Campaña son dimensiones potencialmente reutilizables entre procesos, mientras que Geografía, Cliente y Logística se asocian exclusivamente al proceso de Exportaciones.

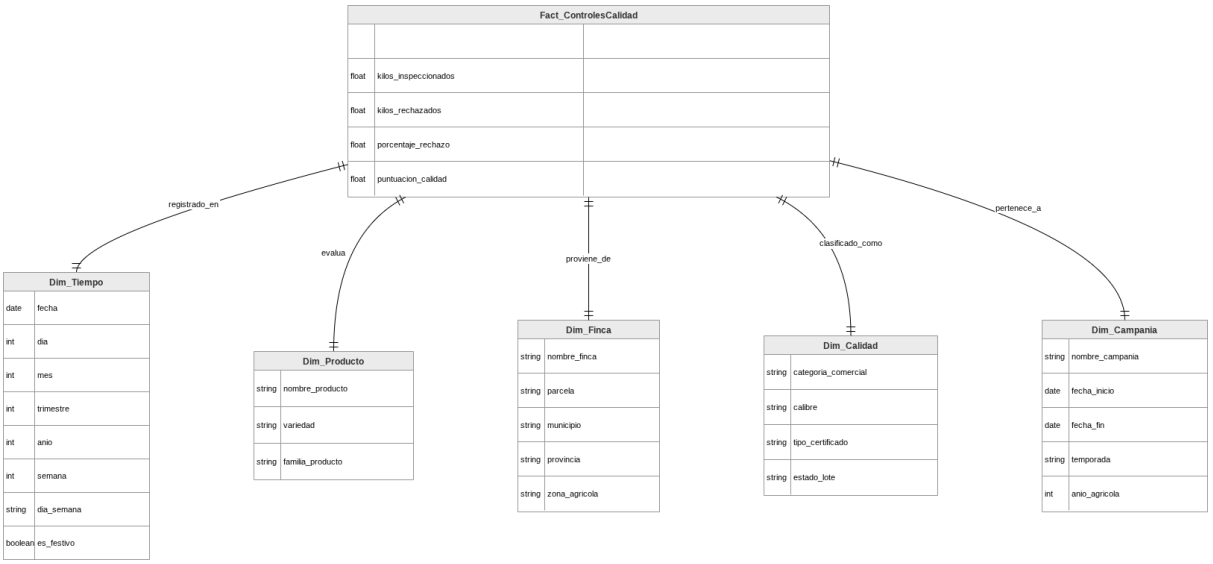
2.7 Diagramas OLAP iniciales

A nivel conceptual se proponen dos diagramas OLAP iniciales:

- Un esquema conceptual centrado en Fact_Exportaciones, con las dimensiones Tiempo, Producto, Geografía, Cliente, Finca, Calidad, Logística y Campaña.



- Un esquema conceptual centrado en Fact_Controles_Calidad, con las dimensiones Tiempo, Producto, Finca, Calidad y Campaña.



Estos diagramas sirven como transición entre el análisis de negocio y el diseño definitivo del modelo dimensional implementado en la sección 4.

3. Diseño de fuentes de datos transaccionales y población inicial

3.1 Introducción

En este apartado se diseñan los cinco esquemas OLTP que simulan los sistemas operacionales de AgroCeuta. Cada departamento mantiene su propio esquema PostgreSQL, con tablas normalizadas, claves primarias, claves foráneas, restricciones y relaciones cruzadas entre departamentos.

El objetivo es crear un entorno realista que posteriormente será integrado en el Data Warehouse mediante procesos ETL.

3.2 Arquitectura de esquemas OLTP

Se crearán los siguientes esquemas en PostgreSQL:

CREATE SCHEMA oltp_produccion;
CREATE SCHEMA oltp_calidad;

CREATE SCHEMA oltp_ventas;
CREATE SCHEMA oltp_logistica;
CREATE SCHEMA oltp_finanzas;

3.3 Diseño por departamento

A continuación se describe la estructura de cada departamento. Para cada uno se indican:

- Diagrama ER conceptual (descripción textual).
- Tablas principales con atributos.
- Relaciones clave.

3.3.1 Departamento Producción Agrícola (oltp_produccion)

Objetivo: registrar la actividad agrícola en campo, incluyendo fincas, parcelas, campañas, cultivos, cosechas y lotes de origen.

Tablas principales:
1. fincas:
finca_id (PK), nombre_finca, superficie_total_hectareas, municipio, provincia, zona_agricola, tipo_suelo, sistema_riego

2. parcelas:
parcela_id (PK), finca_id (FK → fincas), nombre_parcela, superficie_hectareas, cultivo_actual, fecha_ultima_rotacion

3. campanias (campañas)
campania_id (PK), nombre_campania, fecha_inicio, fecha_fin, temporada, anio_agricola, observaciones_climaticas

4. cultivos
cultivo_id (PK), nombre_producto, familia_producto, variedad, ciclo_dias, riego_necesario_litros_hectarea

5. cosechas
cosecha_id (PK), parcela_id (FK → parcelas), cultivo_id (FK → cultivos), campania_id (FK → campanias), fecha_cosecha, kilos_cosechados, rendimiento_kg_hectarea, coste_produccion, merma_campo_porcentaje, numero_recolecciones

6. lotesorigen
lote_id (PK), cosecha_id (FK → cosechas), codigo_lote, kilos_lote, fecha_creacion_lote, estado (en_campo, en_almacen, enviado)

Relaciones:
- fincas 1:N parcelas
- parcelas 1:N cosechas
- cultivos 1:N cosechas
- campanias 1:N cosechas
- cosechas 1:N lotes_origen

Restricciones destacadas:
- superficie_hectareas > 0
- rendimiento_kg_hectarea > 0
- merma_campo_porcentaje BETWEEN 0 AND 100
- fecha_fin > fecha_inicio en campanias

Índices:
- CREATE INDEX idx_cosechas_fecha ON oltp_produccion.cosechas(fecha_cosecha);
- CREATE INDEX idx_lotes_codigo ON oltp_produccion.lotes_origen(codigo_lote);

3.3.2 Departamento Calidad y Trazabilidad (oltp_calidad)

Objetivo: almacenar controles de calidad, calibres, certificados e incidencias asociados a los lotes.

Tablas principales:
1. lotes
lote_id (PK y FK → oltp_produccion.lotes_origen), codigo_lote, producto, kilos_totales, fecha_entrada_almacen

2. controles_calidad
control_id (PK), lote_id (FK → lotes), fecha_control, inspector, kilos_inspeccionados, kilos_rechazados, porcentaje_merma, porcentaje_primera_categoria, porcentaje_segunda_categoria, puntuacion_calidad

3. calibres

calibre_id (PK), lote_id (FK → lotes), calibre (S, M, L, XL), kilos_calibre, porcentaje_sobre_total

4. certificados
certificado_id (PK), lote_id (FK → lotes), tipo_certificado (GlobalGAP, ECO, ISO22000, GRASP), fecha_emision, fecha_caducidad, entidad_certificadora, numero_certificado

5. incidencias_calidad
incidencia_id (PK), lote_id (FK → lotes), fecha_incidencia, tipo_incidencia (plagas, defectos, contaminacion, rechazo_cliente), descripcion, kilos_afectados, accion_correctiva

Relaciones:
- lotes 1:N controles_calidad
- lotes 1:N calibres
- lotes 1:N certificados
- lotes 1:N incidencias_calidad

Restricciones destacadas:
- puntuacion_calidad BETWEEN 0 AND 10
- porcentaje_merma BETWEEN 0 AND 100
- fecha_caducidad > fecha_emision

Índices:
- CREATE INDEX idx_controles_fecha ON oltp_calidad.controles_calidad(fecha_control);
- CREATE INDEX idx_certificados_tipo ON oltp_calidad.certificados(tipo_certificado);

3.3.3 Departamento Ventas Comerciales (oltp_ventas)

Objetivo: gestionar clientes, contratos, pedidos, líneas de pedido y tarifas comerciales.

Tablas principales:
1. clientes
cliente_id (PK), nombre_cliente, cif, tipo_cliente, canal, segmento, pais, ciudad, direccion, codigo_postal, telefono, email, fecha_alta

2. contratos

contrato_id (PK), cliente_id (FK → clientes), fecha_inicio, fecha_fin, condiciones_pago, descuento_general_porcentaje, volumen_minimo_kg, precio_base_kg

3. pedidos
pedido_id (PK), cliente_id (FK → clientes), contrato_id (FK → contratos, nullable), fecha_pedido, fecha_entrega_solicitada, estado (pendiente, confirmado, enviado, entregado, cancelado), observaciones

4. detalle_pedido
detalle_id (PK), pedido_id (FK → pedidos), lote_id (FK → oltp_calidad.lotes), producto, variedad, kilos_solicitados, cajas_solicitadas, precio_unitario_kg, descuento_porcentaje, importe_bruto, importe_neto

5. tarifas
tarifa_id (PK), producto, variedad, calibre, precio_base_kg, fecha_vigencia_inicio, fecha_vigencia_fin

Relaciones:
- clientes 1:N contratos
- clientes 1:N pedidos
- contratos 1:N pedidos
- pedidos 1:N detalle_pedido

Restricciones destacadas:
- cif es UNIQUE
- fecha_entrega_solicitada > fecha_pedido
- importe_neto = ROUND(importe_bruto * (1 - descuento_porcentaje / 100.0), 2)

Índices:
- CREATE INDEX idx_pedidos_fecha ON oltp_ventas.pedidos(fecha_pedido);
- CREATE INDEX idx_detalle_producto ON oltp_ventas.detalle_pedido(producto);

3.3.4 Departamento Logística y Exportación (oltp_logistica)

Objetivo: gestionar transitarios, rutas, puertos, envíos, trámites aduaneros e incidencias logísticas.

Tablas principales:
1. transitarios
transitario_id (PK), nombre_transitario, cif, pais, telefono, email, tipo_transporte, tarifa_base_kg

2. puertos
puerto_id (PK), nombre_puerto, ciudad, pais, tipo, codigo_unlocode

3. rutas_logisticas
ruta_id (PK), transitario_id (FK → transitarios), puerto_salida_id (FK → puertos), puerto_llegada_id (FK → puertos), tipo_transporte, duracion_estimada_dias, coste_base_kg, observaciones

4. envios
envio_id (PK), pedido_id (FK → oltp_ventas.pedidos), ruta_id (FK → rutas_logisticas), transitario_id (FK → transitarios), fecha_envio, fecha_entrega_estimada, fecha_entrega_real, kilos_totales, numero_contenedor, coste_envio_total, coste_por_kg, estado

5. aduanas
aduana_id (PK), envio_id (FK → envios), pais_aduana, fecha_entrada_aduana, fecha_salida_aduana, coste_aranceles, documentacion_completa

6. incidencias_envio
incidencia_envio_id (PK), envio_id (FK → envios), fecha_incidencia, tipo_incidencia, descripcion, coste_adicional, responsable

Relaciones:
- transitarios 1:N rutas_logisticas
- puertos 1:N rutas_logisticas como puerto de salida
- puertos 1:N rutas_logisticas como puerto de llegada
- pedidos 1:N envios
- rutas_logisticas 1:N envios
- transitarios 1:N envios
- envios 1:N aduanas
- envios 1:N incidencias_envio

Restricciones destacadas:
- duracion_estimada_dias > 0
- fecha_entrega_estimada > fecha_envio

- fecha_entrega_real > fecha_envio

Índices:
- CREATE INDEX idx_envios_fecha ON oltp_logistica.envios(fecha_envio);
- CREATE INDEX idx_rutas_transitario ON oltp_logistica.rutas_logisticas(transitario_id);

3.3.5 Departamento Finanzas y Costes (oltp_finanzas)

Objetivo: registrar facturación, cobros, costes de producción por lote, costes logísticos y tipos de cambio.

Tablas principales:
1. facturas
factura_id (PK), pedido_id (FK único → oltp_ventas.pedidos), fecha_emision, fecha_vencimiento, importe_bruto, descuento, impuestos, importe_total, divisa, estado

2. cobros
cobro_id (PK), factura_id (FK → facturas), fecha_cobro, importe_cobrado, metodo_pago, banco, observaciones

3. costes_lote
coste_lote_id (PK), lote_id (FK único → oltp_produccion.lotes_origen), coste_semillas, coste_agua, coste_fertilizantes, coste_mano_obra, coste_maquinaria, coste_total_produccion, coste_por_kg

4. costes_logisticos
coste_logistico_id (PK), envio_id (FK único → oltp_logistica.envios), coste_transporte, coste_aduana, coste_almacenaje, coste_seguro, coste_total_logistica, coste_por_kg

5. tipos_cambio
tipo_cambio_id (PK), divisa_origen, divisa_destino, fecha, tasa_cambio

Relaciones:
- pedidos 1:1 facturas
- facturas 1:N cobros
- lotes_origen 1:1 costes_lote
- envios 1:1 costes_logisticos

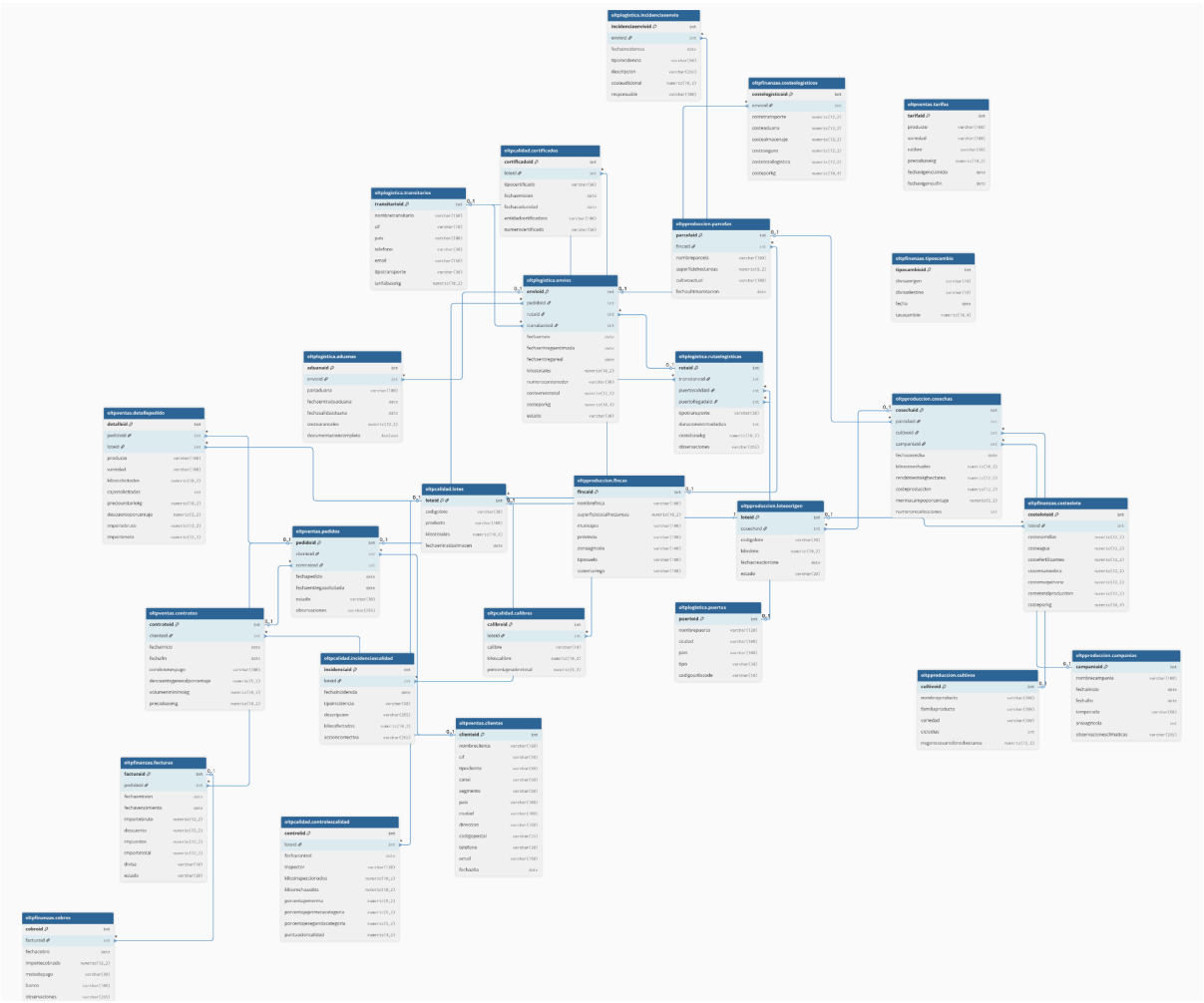
Restricciones destacadas:

- fecha_vencimiento > fecha_emision
- importe_total = ROUND(importe_bruto - descuento + impuestos, 2)
- existe un trigger que valida que fecha_cobro no sea anterior a fecha_emision

Índices:

- CREATE INDEX idx_facturas_fecha ON oltp_finanzas.facturas(fecha_emision);
- CREATE INDEX idx_cobros_fecha ON oltp_finanzas.cobros(fecha_cobro);

Como resultado, este es el diagrama ER completo:



Ante la previsión de que no sea fácil de ver, debido al gran tamaño del diagrama, se subirá este diagrama al repositorio de GitHub mencionado en el Anexo A de esta memoria.

3.4 Relaciones cruzadas entre departamentos

Las relaciones cruzadas entre los esquemas transaccionales son las siguientes:

- oltp_produccion.lotes_origen → oltp_calidad.lotes : relación 1:1, ya que ambos representan el mismo lote en contextos distintos.
- oltp_calidad.lotes → oltp_ventas.detalle_pedido : relación 1:N, ya que un lote puede aparecer en varias líneas de pedido.
- oltp_ventas.pedidos → oltp_logistica.envios : relación 1:N, ya que un pedido puede asociarse a varios envíos.
- oltp_ventas.pedidos → oltp_finanzas.facturas : relación 1:1, ya que pedido_id es único en facturas.
- oltp_produccion.lotes_origen → oltp_finanzas.costes_lote : relación 1:1, ya que lote_id es único en costes_lote.
- oltp_logistica.envios → oltp_finanzas.costes_logisticos : relación 1:1, ya que envio_id es único en costes_logisticos.

3.5 Población de datos sintéticos (DML)

Para poblar la base de datos con registros realistas, he empleado generación asistida para preparar datos sintéticos y después se han ajustado manualmente las sentencias SQL. El procedimiento seguido ha sido el siguiente:

- El script crea primero la estructura de esquemas y tablas OLTP y OLAP.
- A continuación genera los datos de OLTP con generate_series, formulas y joins, garantizando consistencia entre entidades.
- Luego transforma esos datos a modelos dimensionales star y snowflake.
- El diseño permite comparar tiempos de consulta entre OLTP y OLAP con datos coherentes pero artificiales.

Ejemplo de INSERT para produccion.parcelas:

```
INSERT INTO oltp_produccion.parcelas
(finca_id, nombre_parcela, superficie_hectareas, cultivo_actual, fecha_ultima_rotacion)
SELECT
  ((g - 1) % 60) + 1,
  'Parcela_' || LPAD(g::text, 3, '0'),
  ROUND((2.50 + (g % 8) * 0.70)::numeric, 2),
  CASE g % 10
    WHEN 0 THEN 'Tomate'
    WHEN 1 THEN 'Pimiento'
    WHEN 2 THEN 'Pepino'
    WHEN 3 THEN 'Aguacate'
    WHEN 4 THEN 'Fresa'
    WHEN 5 THEN 'Naranja'
    WHEN 6 THEN 'Limon'
    WHEN 7 THEN 'Calabacin'
    WHEN 8 THEN 'Frambuesa'
    ELSE 'Arandano'
  END,
  DATE '2023-01-01' + (g * 6)
FROM generate_series(1, 120) g;
```

Nota: Habrá un único script con toda la creación y población disponible en el repositorio GitHub mencionado en el Anexo A y en la entrega de Prado.

4. Modelo dimensional y ETL básico

Tras el análisis conceptual de la sección 2, he decidido implementar un modelo estrella centrado en el proceso de negocio principal Exportaciones. Esta decisión se justifica porque dicho proceso integra información procedente de producción, calidad, ventas, logística y finanzas, y permite responder de forma directa a las cinco preguntas de negocio planteadas.

Aunque a nivel conceptual se identificó también el proceso Controles de calidad como hecho candidato, en esta primera implementación del Data Warehouse se prioriza un único hecho principal, Fact_Exportaciones, con el objetivo de mantener un esquema estrella simple, fácil de implementar y adecuado para el benchmarking posterior entre OLTP, Star Schema y Snowflake.

La granularidad definitiva de Fact_Exportaciones es una fila por línea de pedido exportada. Cada fila queda contextualizada por las dimensiones Tiempo, Producto, Geografía, Cliente, Finca, Calidad, Logística y Campaña.

Las métricas definitivas de la tabla de hechos son:
- kilos_vendidos
- cajas_vendidas
- importe_bruto
- descuento
- importe_neto
- coste_producto
- coste_logistico
- coste_total
- margen
- precio_medio_kg
- tiempo_entrega_dias

4.1 Metodología de transformación OLTP

Data Warehouse

Para la construcción del Data Warehouse de AgroCeuta se ha seguido una metodología de transformación desde sistemas OLTP hacia un modelo dimensional orientado al análisis. El punto de partida han sido las preguntas de negocio definidas en el Hito 1, ya que estas determinan qué métricas deben analizarse, qué dimensiones son necesarias y cuál debe ser la granularidad de la tabla de hechos.

En este caso, el proceso de negocio central seleccionado es el de las exportaciones, puesto que integra información procedente de ventas, producción, calidad, logística y finanzas. Por ello, se ha optado por una única tabla de hechos, denominada Fact_Exportaciones, con varias dimensiones conformadas que permiten analizar la actividad desde diferentes perspectivas.

La granularidad elegida para la tabla de hechos es una fila por línea de pedido exportada. Esto significa que cada registro de la tabla de hechos representa un producto concreto vendido a un cliente concreto, en una fecha determinada, procedente de una finca específica, asociado a una calidad de lote determinada y enviado mediante una configuración logística concreta.

4.2 Mapeo de fuentes OLTP a dimensiones y hecho

El proceso de mapeo entre OLTP y OLAP se ha realizado identificando qué entidades del sistema transaccional aportan atributos descriptivos para las dimensiones y cuáles proporcionan medidas para la tabla de hechos.

oltp_ventas.clientes alimenta dim_cliente, al contener atributos descriptivos del cliente como nombre, tipo, canal, segmento, país, ciudad, email y teléfono.

oltp_produccion.cultivos alimenta dim_producto, aportando nombre_producto, variedad y familia_producto. Además, durante la carga ETL se derivan atributos comerciales como tipo_envase, formato_comercial y peso_unidad.

oltp_produccion.fincas y oltp_produccion.parcelas alimentan dim_finca, ya que describen el origen productivo de la mercancía.

oltp_calidad.controles_calidad, oltp_calidad.calibres, oltp_calidad.certificados y oltp_produccion.lotes_origen alimentan dim_calidad, proporcionando la categoría comercial, el calibre dominante, el tipo de certificado y el estado del lote.

oltp_logistica.rutas_logisticas, oltp_logistica.transitarios y oltp_logistica.puertos alimentan dim_logistica, al definir el transitario, los puertos de salida y llegada, el tipo de transporte y la ruta logística.

oltp_produccion.campanias alimenta dim_campania, incorporando el contexto agrícola de cada operación.

oltp_ventas.pedidos alimenta dim_tiempo, usando fecha_pedido como referencia temporal principal.

oltp_ventas.clientes también alimenta dim_geografia, a partir de ciudad y pais, mientras que region, zona_comercial y continente se derivan en el proceso ETL.

La tabla de hechos fact_exportaciones se alimenta principalmente de oltp_ventas.detalle_pedido, enriquecida con datos de pedidos, clientes,

envios, costes_logisticos, costes_lote, lotes_origen, cosechas, cultivos, parcelas, fincas y campanias.

Las métricas seleccionadas son kilos_vendidos, cajas_vendidas, importe_bruto, descuento, importe_neto, coste_producto, coste_logistico, coste_total, margen, precio_medio_kg y tiempo_entrega_dias.

4.3 Bus Matrix final

La Bus Matrix final del sistema queda definida de la siguiente manera:

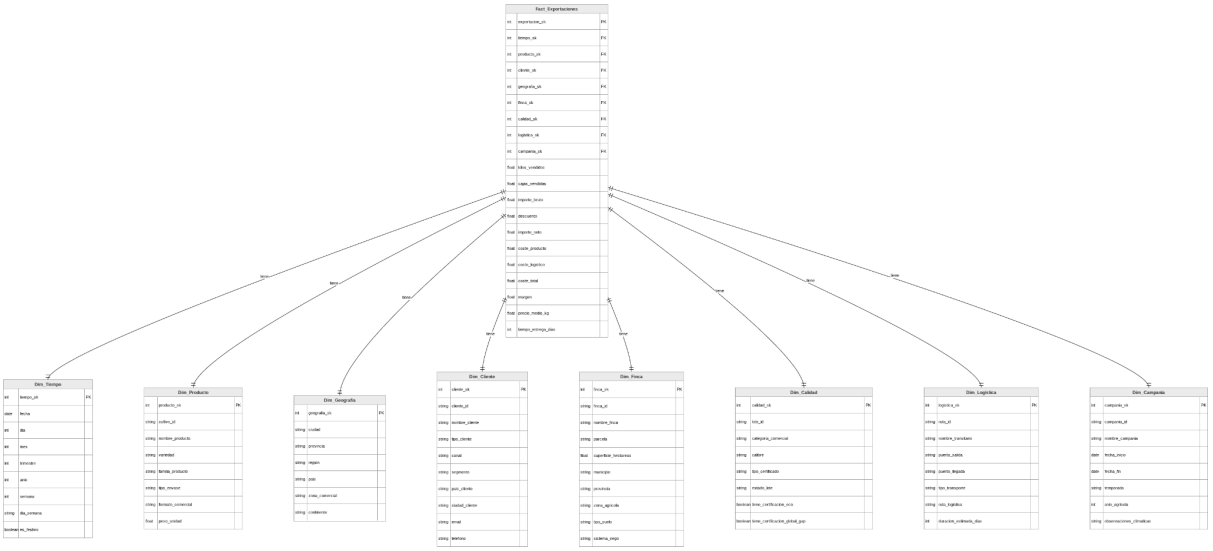
Proceso	Tiempo	Producto	Geografía	Cliente	Finca	Calidad	Logística	Campaña
Exportaciones	X	X	X	X	X	X	X	X

Esta matriz muestra que las dimensiones utilizadas por Fact_Exportaciones son dimensiones potencialmente conformadas o reutilizables en futuras ampliaciones del sistema, como incidencias logísticas o análisis independientes de producción.

4.4 Modelo estrella propuesto

El Data Warehouse se implementa inicialmente mediante un esquema en estrella. En el centro del modelo se sitúa la tabla de hechos fact_exportaciones, conectada con las dimensiones dim_tiempo, dim_producto, dim_geografia, dim_cliente, dim_finca, dim_calidad, dim_logistica y dim_campaña.

La ventaja principal de este enfoque es que simplifica las consultas analíticas y reduce el número de joins necesarios en comparación con un modelo más normalizado. Además, el esquema en estrella se adapta especialmente bien a las operaciones OLAP de agregación, drill-down, roll-up y comparación histórica.



4.5 Scripts de creación del esquema estrella

A continuación se muestra la creación completa del esquema dimensional. Esto va junto al fichero SQL entregado en la práctica (en el repositorio de GitHub y en la entrega de Prado).

```
CREATE SCHEMA olap_star;

CREATE TABLE olap_star.dim_tiempo (
  tiempo_sk SERIAL PRIMARY KEY,
  fecha DATE UNIQUE NOT NULL,
  dia INT NOT NULL,
  mes INT NOT NULL,
  trimestre INT NOT NULL,
  anio INT NOT NULL,
  semana INT,
  dia_semana VARCHAR(20),
  es_festivo BOOLEAN DEFAULT FALSE
);

CREATE TABLE olap_star.dim_campania (
  campania_sk SERIAL PRIMARY KEY,
  campania_id INT NOT NULL,
  nombre_campania VARCHAR(100) NOT NULL,
  fecha_inicio DATE,
  fecha_fin DATE,
  temporada VARCHAR(50),
```

```
  anio_agricola INT,
  observaciones_climaticas VARCHAR(255)
);

CREATE TABLE olap_star.dim_cliente (
  cliente_sk SERIAL PRIMARY KEY,
  cliente_id INT NOT NULL,
  nombre_cliente VARCHAR(150) NOT NULL,
  tipo_cliente VARCHAR(50),
  canal VARCHAR(50),
  segmento VARCHAR(50),
  pais_cliente VARCHAR(100),
  ciudad_cliente VARCHAR(100),
  email VARCHAR(150),
  telefono VARCHAR(30)
);

CREATE TABLE olap_star.dim_geografia (
  geografia_sk SERIAL PRIMARY KEY,
  ciudad VARCHAR(100),
  provincia VARCHAR(100),
  region VARCHAR(100),
  pais VARCHAR(100),
  zona_comercial VARCHAR(100),
  continente VARCHAR(100)
);

CREATE TABLE olap_star.dim_finca (
  finca_sk SERIAL PRIMARY KEY,
  finca_id INT NOT NULL,
  nombre_finca VARCHAR(150),
  parcela VARCHAR(100),
  superficie_hectareas NUMERIC(10,2),
  municipio VARCHAR(100),
  provincia VARCHAR(100),
  zona_agricola VARCHAR(100),
  tipo_suelo VARCHAR(100),
  sistema_riego VARCHAR(100)
);

CREATE TABLE olap_star.dim_producto (
```

```
  producto_sk SERIAL PRIMARY KEY,
  cultivo_id INT NOT NULL,
  nombre_producto VARCHAR(100),
  variedad VARCHAR(100),
  familia_producto VARCHAR(100),
  tipo_envase VARCHAR(100),
  formato_comercial VARCHAR(100),
  peso_unidad NUMERIC(10,2)
);

CREATE TABLE olap_star.dim_calidad (
  calidad_sk SERIAL PRIMARY KEY,
  lote_id INT NOT NULL,
  categoria_comercial VARCHAR(50),
  calibre VARCHAR(20),
  tipo_certificado VARCHAR(100),
  estado_lote VARCHAR(50),
  tiene_certificacion_eco BOOLEAN,
  tiene_certificacion_global_gap BOOLEAN
);

CREATE TABLE olap_star.dim_logistica (
  logistica_sk SERIAL PRIMARY KEY,
  ruta_id INT NOT NULL,
  nombre_transitario VARCHAR(150),
  puerto_salida VARCHAR(100),
  puerto_llegada VARCHAR(100),
  tipo_transporte VARCHAR(50),
  ruta_logistica VARCHAR(150),
  duracion_estimada_dias INT
);

CREATE TABLE olap_star.fact_exportaciones (
  exportacion_sk SERIAL PRIMARY KEY,
  tiempo_sk INT NOT NULL REFERENCES olap_star.dim_tiempo(tiempo_sk),
  producto_sk INT NOT NULL REFERENCES olap_star.dim_producto(producto_sk),
  cliente_sk INT NOT NULL REFERENCES olap_star.dim_cliente(cliente_sk),
  geografia_sk INT NOT NULL REFERENCES olap_star.dim_geografia(geografia_sk),
```

```
  finca_sk INT NOT NULL REFERENCES olap_star.dim_finca(finca_sk),
  calidad_sk INT NOT NULL REFERENCES olap_star.dim_calidad(calidad_sk),
  logistica_sk INT NOT NULL REFERENCES olap_star.dim_logistica(logistica_sk),
  campania_sk INT NOT NULL REFERENCES olap_star.dim_campania(campania_sk),
  kilos_vendidos NUMERIC(12,2) NOT NULL,
  cajas_vendidas INT NOT NULL,
  importe_bruto NUMERIC(12,2) NOT NULL,
  descuento NUMERIC(12,2) NOT NULL,
  importe_neto NUMERIC(12,2) NOT NULL,
  coste_producto NUMERIC(12,2) NOT NULL,
  coste_logistico NUMERIC(12,2) NOT NULL,
  coste_total NUMERIC(12,2) NOT NULL,
  margen NUMERIC(12,2) NOT NULL,
  precio_medio_kg NUMERIC(12,4) NOT NULL,
  tiempo_entrega_dias INT NOT NULL
);
```

4.6 Carga de dimensiones

La carga de las dimensiones se realiza a partir de selecciones (algunas DISTINCT) sobre las tablas OLTP, extrayendo únicamente atributos descriptivos y manteniendo claves sustitutas en el entorno dimensional. A modo de ejemplo, se muestra la carga básica de varias dimensiones.

```
INSERT INTO olap_star.dim_tiempo (fecha, dia, mes, trimestre, anio, semana, dia_semana)
SELECT DISTINCT
   p.fecha_pedido,
   EXTRACT(DAY FROM p.fecha_pedido)::INT,
   EXTRACT(MONTH FROM p.fecha_pedido)::INT,
   EXTRACT(QUARTER FROM p.fecha_pedido)::INT,
   EXTRACT(YEAR FROM p.fecha_pedido)::INT,
   EXTRACT(WEEK FROM p.fecha_pedido)::INT,
   TO_CHAR(p.fecha_pedido, 'Day')
FROM oltp_ventas.pedidos p;

INSERT INTO olap_star.dim_campania (campania_id, nombre_campania, fecha_inicio, fecha_fin, temporada, anio_agricola, observaciones_climaticas)
```

```
SELECT
   c.campania_id, c.nombre_campania, c.fecha_inicio, c.fecha_fin,
   c.temporada, c.anio_agricola, c.observaciones_climaticas
FROM oltp_produccion.campanias c;

INSERT INTO olap_star.dim_cliente (cliente_id, nombre_cliente, tipo_cliente,
canal, segmento, pais_cliente, ciudad_cliente, email, telefono)
SELECT
   c.cliente_id, c.nombre_cliente, c.tipo_cliente, c.canal, c.segmento,
   c.pais, c.ciudad, c.email, c.telefono
FROM oltp_ventas.clientes c;

INSERT INTO olap_star.dim_geografia (ciudad, provincia, region, pais,
zona_comercial, continente)
SELECT DISTINCT
   c.ciudad,
   NULL,
   NULL,
   c.pais,
   c.pais,
   CASE
         WHEN c.pais IN ('España','Portugal','Francia','Italia','Alemania','Países
Bajos') THEN 'Europa'
      ELSE 'Otros'
   END
FROM oltp_ventas.clientes c;
```

4.7 ETL básico de la tabla de hechos

La carga de fact_exportaciones se realiza integrando información de ventas, producción, calidad, logística y finanzas en un único registro analítico por línea de pedido. Este proceso permite transformar datos operacionales en medidas agregables aptas para consulta multidimensional.

```
INSERT INTO olap_star.fact_exportaciones
(tiempo_sk, producto_sk, cliente_sk, geografia_sk, finca_sk, calidad_sk,
logistica_sk, campania_sk,
kilos_vendidos, cajas_vendidas, importe_bruto, descuento, importe_neto,
coste_producto, coste_logistico, coste_total, margen, precio_medio_kg,
tiempo_entrega_dias)
```

```
SELECT
  dt.tiempo_sk,
  dp.producto_sk,
  dc.cliente_sk,
  dg.geografia_sk,
  df.finca_sk,
  dq.calidad_sk,
  dl.logistica_sk,
  dca.campania_sk,
  d.kilos_solicitados,
  d.cajas_solicitadas,
  d.importe_bruto,
  ROUND((d.importe_bruto - d.importe_neto)::numeric, 2) AS descuento,
  d.importe_neto,
        ROUND((d.kilos_solicitados * cl.coste_por_kg)::numeric, 2) AS
coste_producto,
       ROUND((d.kilos_solicitados * clog.coste_por_kg)::numeric, 2) AS
coste_logistico,
     ROUND(((d.kilos_solicitados * cl.coste_por_kg) + (d.kilos_solicitados *
clog.coste_por_kg))::numeric, 2) AS coste_total,
      ROUND((d.importe_neto - ((d.kilos_solicitados * cl.coste_por_kg) +
(d.kilos_solicitados * clog.coste_por_kg)))::numeric, 2) AS margen,
        ROUND((d.importe_neto / d.kilos_solicitados)::numeric, 4) AS
precio_medio_kg,
  (e.fecha_entrega_real - e.fecha_envio) AS tiempo_entrega_dias
FROM oltp_ventas.detalle_pedido d
JOIN oltp_ventas.pedidos p ON p.pedido_id = d.pedido_id
JOIN oltp_ventas.clientes c ON c.cliente_id = p.cliente_id
JOIN oltp_logistica.envios e ON e.pedido_id = p.pedido_id
JOIN oltp_finanzas.costes_logisticos clog ON clog.envio_id = e.envio_id
JOIN oltp_finanzas.costes_lote cl ON cl.lote_id = d.lote_id
JOIN oltp_produccion.lotes_origen lo ON lo.lote_id = d.lote_id
JOIN oltp_produccion.cosechas co ON co.cosecha_id = lo.cosecha_id
JOIN oltp_produccion.cultivos cu ON cu.cultivo_id = co.cultivo_id
JOIN oltp_produccion.parcelas pa ON pa.parcela_id = co.parcela_id
JOIN oltp_produccion.fincas fi ON fi.finca_id = pa.finca_id
JOIN oltp_produccion.campanias ca ON ca.campania_id = co.campania_id
JOIN olap_star.dim_tiempo dt ON dt.fecha = p.fecha_pedido
JOIN olap_star.dim_producto dp ON dp.cultivo_id = cu.cultivo_id
JOIN olap_star.dim_cliente dc ON dc.cliente_id = c.cliente_id
```

```
JOIN olap_star.dim_geografia dg ON dg.pais = c.pais AND dg.ciudad = c.ciudad
JOIN olap_star.dim_finca df ON df.finca_id = fi.finca_id AND df.parcela = pa.nombre_parcela
JOIN olap_star.dim_calidad dq ON dq.lote_id = d.lote_id
JOIN olap_star.dim_logistica dl ON dl.ruta_id = e.ruta_id
JOIN olap_star.dim_campania dca ON dca.campania_id = ca.campania_id;
```

4.8 Propuesta de modelo snowflake

Para poder comparar el rendimiento del esquema estrella con una versión más normalizada, se ha creado un esquema snowflake sobre la dimensión de producto. En este caso, dim_producto se descompone en una tabla de familias de producto y otra tabla de productos que referencia a la familia, aumentando el número de joins pero reduciendo redundancia.

```
CREATE SCHEMA olap_snow;

CREATE TABLE olap_snow.dim_familia_producto (
    familia_sk SERIAL PRIMARY KEY,
    familia_producto VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE olap_snow.dim_producto (
   producto_sk SERIAL PRIMARY KEY,
   cultivo_id INT NOT NULL,
   nombre_producto VARCHAR(100),
   variedad VARCHAR(100),
familia_sk INT NOT NULL REFERENCES olap_snow.dim_familia_producto(familia_sk),
   tipo_envase VARCHAR(100),
   formato_comercial VARCHAR(100),
   peso_unidad NUMERIC(10,2)
);
```

La tabla de hechos del esquema snowflake mantiene la misma granularidad y las mismas métricas que en el modelo estrella, pero las consultas sobre producto requieren al menos un join adicional.

5. Benchmarking y evaluación

5.1 Objetivo del benchmarking

El objetivo de esta fase es comparar el comportamiento de las consultas analíticas en tres contextos distintos: modelo OLTP, modelo estrella y modelo snowflake. Con ello se pretende comprobar que el diseño dimensional optimiza las consultas analíticas frente al modelo transaccional y que el esquema estrella suele comportarse mejor que el snowflake por requerir menos joins.

5.2 Consultas que resuelven las preguntas de negocio

Al principio de la práctica, se definieron cinco preguntas de negocio representativas. Estas consultas son las que permiten resolverlas. Con el fin de no agrandar de forma excesiva e innecesaria la memoria, solo se incluirán las consultas definidas para el modelo OLTP; las consultas del modelo estrella y copo de nieve están definidas dentro del script de benchmark, que se puede consultar en el repositorio de GitHub mencionado en el Anexo A de esta memoria.

Consulta 1. Ingresos, costes y margen por año, trimestre, campaña, producto, variedad y país.

```
SELECT
        EXTRACT(YEAR FROM p.fecha_pedido) AS anio,
        EXTRACT(QUARTER FROM p.fecha_pedido) AS trimestre,
        ca.nombre_campania,
        cu.familia_producto,
        cu.nombre_producto,
        cu.variedad,
        c.pais,
        SUM(d.importe_neto) AS ingresos,
        SUM(d.kilos_solicitados * cl.coste_por_kg) AS costeproducto,
        SUM(d.kilos_solicitados * clog.coste_por_kg) AS costelogistico,
                SUM(d.importe_neto - (d.kilos_solicitados * cl.coste_por_kg) -
(d.kilos_solicitados * clog.coste_por_kg)) AS margen
    FROM oltp_ventas.detalle_pedido d
    JOIN oltp_ventas.pedidos p ON p.pedido_id = d.pedido_id
    JOIN oltp_ventas.clientes c ON c.cliente_id = p.cliente_id
    JOIN oltp_logistica.envios e ON e.pedido_id = p.pedido_id
    JOIN oltp_finanzas.costes_logisticos clog ON clog.envio_id = e.envio_id
    JOIN oltp_finanzas.costes_lote cl ON cl.lote_id = d.lote_id
    JOIN oltp_produccion.lotes_origen lo ON lo.lote_id = d.lote_id
    JOIN oltp_produccion.cosechas co ON co.cosecha_id = lo.cosecha_id
    JOIN oltp_produccion.cultivos cu ON cu.cultivo_id = co.cultivo_id
            JOIN oltp_produccion.campanias ca ON ca.campania_id =
co.campania_id
    GROUP BY
        EXTRACT(YEAR FROM p.fecha_pedido),
        EXTRACT(QUARTER FROM p.fecha_pedido),
        ca.nombre_campania,
        cu.familia_producto,
        cu.nombre_producto,
        cu.variedad,
        c.pais;
```

Consulta 2. Kilos vendidos y margen por finca, producto y campaña.

```
SELECT
        fi.nombre_finca,
        ca.nombre_campania,
        cu.familia_producto,
        cu.nombre_producto,
```

```
    SUM(d.kilos_solicitados) AS kilos_vendidos,
        SUM(d.importe_neto - (d.kilos_solicitados * cl.coste_por_kg) - (d.kilos_solicitados * clog.coste_por_kg)) AS margen
  FROM oltp_ventas.detalle_pedido d
  JOIN oltp_ventas.pedidos p ON p.pedido_id = d.pedido_id
  JOIN oltp_logistica.envios e ON e.pedido_id = p.pedido_id
  JOIN oltp_finanzas.costes_logisticos clog ON clog.envio_id = e.envio_id
  JOIN oltp_finanzas.costes_lote cl ON cl.lote_id = d.lote_id
  JOIN oltp_produccion.lotes_origen lo ON lo.lote_id = d.lote_id
  JOIN oltp_produccion.cosechas co ON co.cosecha_id = lo.cosecha_id
  JOIN oltp_produccion.parcelas pa ON pa.parcela_id = co.parcela_id
  JOIN oltp_produccion.fincas fi ON fi.finca_id = pa.finca_id
  JOIN oltp_produccion.cultivos cu ON cu.cultivo_id = co.cultivo_id
        JOIN oltp_produccion.campanias ca ON ca.campania_id = co.campania_id
  GROUP BY
    fi.nombre_finca,
    ca.nombre_campania,
    cu.familia_producto,
    cu.nombre_producto;
```

Consulta 3. Kilos vendidos, precio medio y margen por cliente y país.

```
  SELECT
    cu.familia_producto,
    c.nombre_cliente,
    c.pais,
    SUM(d.kilos_solicitados) AS kilos_vendidos,
        AVG(d.importe_neto / NULLIF(d.kilos_solicitados, 0)) AS precio_medio_kg,
        SUM(d.importe_neto - (d.kilos_solicitados * cl.coste_por_kg) - (d.kilos_solicitados * clog.coste_por_kg)) AS margen
  FROM oltp_ventas.detalle_pedido d
  JOIN oltp_ventas.pedidos p ON p.pedido_id = d.pedido_id
  JOIN oltp_ventas.clientes c ON c.cliente_id = p.cliente_id
  JOIN oltp_logistica.envios e ON e.pedido_id = p.pedido_id
  JOIN oltp_finanzas.costes_logisticos clog ON clog.envio_id = e.envio_id
  JOIN oltp_finanzas.costes_lote cl ON cl.lote_id = d.lote_id
  JOIN oltp_produccion.lotes_origen lo ON lo.lote_id = d.lote_id
  JOIN oltp_produccion.cosechas co ON co.cosecha_id = lo.cosecha_id
```

```
    JOIN oltp_produccion.cultivos cu ON cu.cultivo_id = co.cultivo_id
    GROUP BY
      cu.familia_producto,
      c.nombre_cliente,
      c.pais;
```

Consulta 4. Tiempo medio de entrega y coste logístico medio por transitario y ruta logística.

```
SELECT
      cu.familia_producto,
      tr.nombre_transitario,
      ps.nombre_puerto AS puerto_salida,
      pl.nombre_puerto AS puerto_llegada,
      AVG(e.fecha_entrega_real - e.fecha_envio) AS tiempo_medio_entrega,
      AVG(clog.coste_por_kg) AS costelogisticomediokg
    FROM oltp_logistica.envios e
    JOIN oltp_ventas.pedidos p ON p.pedido_id = e.pedido_id
    JOIN oltp_ventas.detalle_pedido d ON d.pedido_id = p.pedido_id
    JOIN oltp_produccion.lotes_origen lo ON lo.lote_id = d.lote_id
    JOIN oltp_produccion.cosechas co ON co.cosecha_id = lo.cosecha_id
    JOIN oltp_produccion.cultivos cu ON cu.cultivo_id = co.cultivo_id
    JOIN oltp_logistica.rutas_logisticas r ON r.ruta_id = e.ruta_id
    JOIN oltp_logistica.transitarios tr ON tr.transitario_id = r.transitario_id
    JOIN oltp_logistica.puertos ps ON ps.puerto_id = r.puerto_salida_id
    JOIN oltp_logistica.puertos pl ON pl.puerto_id = r.puerto_llegada_id
    JOIN oltp_finanzas.costes_logisticos clog ON clog.envio_id = e.envio_id
    GROUP BY
      cu.familia_producto,
      tr.nombre_transitario,
      ps.nombre_puerto,
      pl.nombre_puerto;
```

Consulta 5. Precio medio y margen total por categoría comercial, calibre y tipo de certificado.

```
WITH calibredominante AS (
      SELECT lote_id AS lote_id, calibre
      FROM (
        SELECT
          lote_id,
```

```
            calibre,
            kilos_calibre AS kiloscalibre,
                    ROW_NUMBER() OVER (PARTITION BY lote_id ORDER BY
kilos_calibre DESC, calibre) AS rn
          FROM oltp_calidad.calibres
        ) x
        WHERE rn = 1
      )
      SELECT
        cu.familia_producto,
        CASE
          WHEN cc.porcentaje_primera_categoria >= 70 THEN 'Primera'
          ELSE 'Segunda'
        END AS categoriacomercial,
        cd.calibre,
        cert.tipo_certificado,
                    AVG(d.importe_neto / NULLIF(d.kilos_solicitados, 0)) AS
preciomediokg,
                SUM(d.importe_neto - (d.kilos_solicitados * cl.coste_por_kg) -
(d.kilos_solicitados * clog.coste_por_kg)) AS margentotal
      FROM oltp_ventas.detalle_pedido d
      JOIN oltp_ventas.pedidos p ON p.pedido_id = d.pedido_id
      JOIN oltp_logistica.envios e ON e.pedido_id = p.pedido_id
      JOIN oltp_finanzas.costes_logisticos clog ON clog.envio_id = e.envio_id
      JOIN oltp_finanzas.costes_lote cl ON cl.lote_id = d.lote_id
      JOIN oltp_produccion.lotes_origen lo ON lo.lote_id = d.lote_id
      JOIN oltp_produccion.cosechas co ON co.cosecha_id = lo.cosecha_id
      JOIN oltp_produccion.cultivos cu ON cu.cultivo_id = co.cultivo_id
      JOIN oltp_calidad.controles_calidad cc ON cc.lote_id = d.lote_id
      JOIN calibredominante cd ON cd.lote_id = d.lote_id
      JOIN oltp_calidad.certificados cert ON cert.lote_id = d.lote_id
      GROUP BY
        cu.familia_producto,
        CASE
          WHEN cc.porcentaje_primera_categoria >= 70 THEN 'Primera'
          ELSE 'Segunda'
        END,
        cd.calibre,
        cert.tipo_certificado;
```

5.3 Ejemplo de roll-up y drill-down

Con el fin de demostrar la capacidad analítica del modelo dimensional se han incluido dos operaciones OLAP sobre la dimensión tiempo.

La operación de roll-up consiste en agregar la información a un nivel superior de la jerarquía temporal. En este caso, se parte del nivel detallado de las exportaciones y se agrupan los ingresos por año, obteniendo una visión global de la evolución del negocio.

La operación de drill-down consiste en desagregar la información hacia un nivel más detallado de la misma jerarquía. En este caso, se pasa del nivel anual al nivel trimestral, permitiendo identificar con mayor precisión la distribución temporal de los ingresos dentro de cada ejercicio.

SQL de Roll-up

```
SELECT
   t.anio,
   SUM(f.importe_neto) AS ingresos_totales
FROM olap_star.fact_exportaciones f
JOIN olap_star.dim_tiempo t ON t.tiempo_sk = f.tiempo_sk
GROUP BY t.anio
ORDER BY t.anio;
```

SQL de Drill-down

```
SELECT
   t.anio,
   t.trimestre,
   SUM(f.importe_neto) AS ingresos_totales
FROM olap_star.fact_exportaciones f
JOIN olap_star.dim_tiempo t ON t.tiempo_sk = f.tiempo_sk
GROUP BY t.anio, t.trimestre
ORDER BY t.anio, t.trimestre;
```

5.4 Script Python de benchmarking

Para automatizar la comparación entre OLTP, STAR y SNOWFLAKE se ha preparado un script en Python que ejecuta cada consulta varias veces. En

esta práctica, cada consulta se ha ejecutado por defecto 20 veces, aunque el script permite modificar el número si se desea. Calcula el tiempo medio y la desviación estándar. Esta automatización permite obtener una comparación objetiva del rendimiento entre las tres arquitecturas.

El script se puede obtener al completo en el repositorio de GitHub, cuyo enlace está en el Anexo A de este documento.

5.5 Resultados del benchmarking

Los resultados obtenidos se resumen en la siguiente tabla:

Consulta	OLTP (ms)	STAR (ms)	SNOWFLAKE (ms)	Mejora STAR VS OLTP	Mejora STAR vs SNOW
Query 1	2.299	0.930	1.123	59.55%	17.19%
Query 2	1.625	0.472	0.651	70.95%	27.50%
Query 3	1.667	0.512	0.668	69.29%	23.35%
Query 4	1.357	0.535	0.590	60.57%	9.32%
Query 5	2.489	0.373	0.497	85.01%	24.95%

5.6 Interpretación de resultados

Los resultados obtenidos confirman el comportamiento esperado de los tres modelos evaluados. En las cinco consultas, el modelo OLTP presenta los peores tiempos medios, mientras que el esquema estrella obtiene el mejor rendimiento y el modelo snowflake queda en una posición intermedia.

La mejora del esquema estrella frente al modelo OLTP oscila entre aproximadamente el 59% y el 85%. Esto se debe a que el modelo OLTP obliga a reconstruir el contexto analítico mediante múltiples joins entre tablas normalizadas, mientras que en el esquema estrella las medidas ya están centralizadas en una tabla de hechos conectada a dimensiones desnormalizadas.

Por su parte, el modelo snowflake mantiene un rendimiento muy próximo al esquema estrella, pero normalmente ligeramente peor en todas las consultas. Este comportamiento era esperable, ya que la normalización parcial de dimensiones introduce joins adicionales. En consecuencia, para el caso AgroCeuta, el esquema estrella se confirma como la alternativa más adecuada para la explotación analítica principal.

Como nota adicional, cabe destacar que con estos tiempos tan bajos es posible que el propio Python interfiera en los resultados obtenidos en el benchmark. Además, la normalización realizada en la práctica puede que no sea tan efectiva en determinadas consultas, haciendo que el modelo de copo de nieve pueda rendir muy similar o incluso mejor que el esquema estrella en alguna ejecución suelta.

Para minimizar estos efectos y sacar un resultado más claro, los datos mostrados aquí se han calculado como media de 20 ejecuciones del script de benchmark.

6. Conclusiones

La práctica ha permitido recorrer el ciclo completo de diseño de un sistema multidimensional, desde el análisis de un sistema real en producción hasta el diseño de un entorno OLTP, su transformación a Data Warehouse y la evaluación del rendimiento de las consultas analíticas. En el caso planteado, la empresa AgroCeuta presenta una operativa suficientemente compleja como para justificar la integración de datos de producción, calidad, ventas, logística y finanzas dentro de un único sistema analítico.

El modelo dimensional propuesto, basado en una única tabla de hechos de exportaciones y varias dimensiones conformadas, resulta adecuado para responder a las preguntas estratégicas definidas inicialmente. Además, la comparación entre OLTP, STAR y SNOWFLAKE permite comprobar en la práctica que las estructuras orientadas a análisis mejoran de forma significativa la explotación de información frente a los sistemas transaccionales.

Como trabajo futuro, podrían incorporarse nuevas tablas de hechos orientadas a incidencias logísticas, devoluciones o control histórico de dimensiones mediante técnicas Slowly Changing Dimensions de tipo 2. No

obstante, para los objetivos de esta práctica, el sistema diseñado cumple con los requisitos fundamentales de integración, modelado dimensional, ETL básico y benchmarking analítico.

Anexo A. Archivos adjuntos entregados

Junto con este documento se adjunta un enlace a un repositorio GitHub que contiene todo lo relacionado con la realización de la práctica:

- Los diagramas OLAP y ER.
- Un script SQL unificado que incluye:
    - La creación de esquemas OLTP.
    - La creación del esquema OLAP en estrella.
    - La variante snowflake.
    - La carga de datos.
    - ETL básico.
- El script Python utilizado para el benchmarking.

Enlace al repositorio de GitHub: https://github.com/patxisantos/sm-p1